

Algorithms in C++: Programming Assignment 2

1. Objective

Your goal is to write a program to find “good” prime numbers using the sieve of Eratosthenes. A “good” prime number is a number which:

- Is a prime number
- Is between a given lower and upper bound
- Does not contain a given digit

2. Problem

You are to create a C++ project called sieve, a C++ source file called sieve.cpp, and struct inside it called PrimesEratosthenesSieve that finds “good” prime numbers. The program asks the user for a lower bound, an upper bound, and a bad digit, and then finds “good” primes which are greater than or equal to the lower bound and less than or equal to the upper bound and do not contain the bad digit. When there is only one line of “good” prime numbers in the output, the numbers should be displayed with one space between each (and no space at the end). When there are multiple lines of output, the numbers should be right-aligned to the width of the largest good prime. See below.

```
***** Sieve of Eratosthenes (MODIFIED) *****
Search for good primes between: 0 200 4
Bad digit: 4
Number of good primes found: 42
Good primes between 0 and 200:
  2  3  5  7 11 13 17 19 23 29 31 37 53 59 61 67 71 73 79 83
 89 97 101 103 107 109 113 127 131 137 139 151 157 163 167 173 179 181 191 193
197 199
```

```
***** Sieve of Eratosthenes (MODIFIED) *****
Search for good primes between: 100 200 3
Bad digit: 3
Number of good primes found: 13
Good primes between 100 and 200:
101 107 109 127 149 151 157 167 179 181 191 197 199
```

3. Tips

1. Use the template file found in Canvas. The framework for the program is there, and you will need to fill in the methods. You MUST use the given members and methods to solve the problem.
2. You MUST dynamically allocate the **primes_** bool array. Don't forget to free any dynamically allocated memory.
3. If a function/method is inline, the compiler places a copy of the code of that function/method at each point where it is called at compile time. inline is used for efficiency with short (one-line) functions/methods
4. Look up setw in the iomanip library to set the width of a field that is to be outputted.
5. #include for the sqrt function.

6. You may have up to 80 characters on a line. If you cannot fit all the primes on one line, you should wrap around to the next line. To find the width of the maximum prime value and how many primes you can fit on a row, use the following code:
7. `const int max_prime_width = num_digits(max_prime_);`
`const int primes_per_row = 80 / (max_prime_width + 1);`
8. Be sure to comment your code and put your name and Stevens pledge at the top.
9. Make sure your code works with the test script.
10. The algorithm for the sieve is found in the pseudocode below.

***** Sieve of Eratosthenes *****

Input: an integer $n > 1$

Let `is_prime` be an array of bool values, indexed by integers 2 to n , initially all set to true.

```
for i = 2, 3, 4, ..., while  $i \leq \sqrt{n}$ :  
    if is_prime[i] is true:  
        for  $j = i^2, i^2 + i, i^2 + 2i, \dots$ , while  $j \leq n$ :  
            is_prime[j] = false
```

Now all i such that `is_prime[i]` is true are prime.